Setting up a C compiler

CSCI 1101 Programming Principles I · Fall 2026

This guide gets your laptop ready to write and run C programs. Do it once, at home, before Tuesday's class. It takes between 10 and 30 minutes. If something does not work, do not spend hours on it: post the exact error in the *Questions and answers* forum on Blackboard, or bring the laptop to class with the error on the screen.

You need two things:

1. **A compiler.** This is the program that turns your C code into a program the computer can run. Its name is usually `gcc`.
2. **An editor.** This is where you type your code. We use **Visual Studio Code**, also called VS Code. It is free.

Sections 2 to 7 are the compiler. Pick **one** of them for your computer. Section 8 is VS Code, the same for everyone. Section 9 checks that everything works.

1. Pick one

Your computer	Do this	Time	You need
Windows	Option A: the course installer. One zip file, or one line to paste.	10–20 min	Internet. No admin rights needed.
Windows	Option B: WSL (Linux inside Windows).	20–30 min	Internet, admin rights, a restart.
Windows	Option C: Visual Studio (Microsoft's compiler).	30–60 min	Internet, admin rights, about 10 GB of disk.
Mac	One command in Terminal.	10 min	Internet.
Linux	One command in the terminal.	5 min	Internet, your password.
Any, nothing to install	CoCalc in the browser.	5 min	Internet, an email address.

If you are on Windows and unsure, take **Option A**. If your laptop is broken, borrowed, or you cannot install anything on it, use **CoCalc**. It is enough for this whole course.

2. Windows, option A: the course installer

The installer is a file called `install-gcc-user.bat`. It downloads the compiler and sets it up for you. You do not need to be an administrator. It ends in `.bat`, which means "a list of commands for Windows". Windows runs it when you double-click it.

There are three ways to get it onto your computer. **Way 1 is the easiest.** Way 2 is for anyone who cannot find or open the downloaded file. Way 3 is the internet link, which needs one extra step that catches many people.

Way 1: the zip file on Blackboard

1. On Blackboard, open *Course information* and click `install-gcc-user.zip`. Your browser saves it in your *Downloads* folder.
2. Open your *Downloads* folder. (Press the Windows key and the letter E together to open a file window, then click **Downloads** on the left.)
3. Double-click `install-gcc-user.zip`. A window opens showing one file inside.
4. Click **Extract all** at the top, then **Extract**. A new folder opens with the file `install-gcc-user` in it. Its icon is a small window with a gear, not a notepad.
5. Double-click that file. Go to **"What happens when it runs"** below.

Way 2: one line, no download

1. Press the Windows key, type `cmd`, press Enter. A black window opens.
2. Copy this whole line (select it, then Ctrl + C):

```
curl -L -o "%TEMP%\install-gcc-user.bat" https://gist.githubusercontent.com/Capta1n-n9m0/1445e228be0f4274a01b7ad10e433712/raw/install-gcc-user.bat && "%TEMP%\install-gcc-user.bat"
```

1. Click inside the black window and paste it: press Ctrl + V, or right-click once. Press Enter.
2. The line fetches the installer from the internet and starts it in the same window. Go to **"What happens when it runs"** below. With this way the window stays open at the end, so you can read the last message.

Way 3: the internet link

The installer is also at this address:

```
https://gist.githubusercontent.com/Capta1n-n9m0/1445e228be0f4274a01b7ad10e433712/raw/install-gcc-user.bat
```

If you open it in a browser, you see the text of the installer. **Do not just press "Save".** The browser will save it as a text file called `install-gcc-user.txt`, and double-clicking that opens Notepad instead of running it. Do this instead:

1. Open the address. Press **Ctrl + S**. A save window opens.
2. In the box **Save as type**, choose **All files**. This box is the important one.
3. In the box **File name**, type exactly `install-gcc-user.bat`.
4. Click **Save**. Then open *Downloads* and double-click the file.

If you already saved it and it opens in Notepad, the file is called `.txt`. Rename it:

1. Open *Downloads*. Windows hides the ends of file names, so first make them visible. On Windows 11: click **View** at the top, then **Show**, then tick **File name extensions**. On Windows 10: click the **View** tab, then tick **File name extensions**.
2. Now the file shows as `install-gcc-user.txt`. Right-click it, choose **Rename**, change `.txt` to `.bat`, press Enter.
3. Windows warns that the file might become unusable. Click **Yes**.
4. The icon changes to a small window with a gear. Double-click it.

What happens when it runs

The black window. A black window opens, or the one you already had fills with text. Lines like `Checking PowerShell...` and `Installing MSYS2...` scroll by. This is normal. Do not close it.

A blue box "Windows protected your PC". Windows may show this the first time you double-click the file (Ways 1 and 3). Click **More info**, then **Run anyway**. *(This warning could not be tested on a clean laptop yet. If you see something different, take a photo and post it in the forum.)*

Questions it can ask. Most of the time it asks nothing. It can ask these; the answer is always **Y**:

- `Download and install PowerShell 7 now? [Y/N]:` Press **Y**. Only if your Windows is missing a tool the installer needs.
- `Attempt to install or repair WinGet now? [Y/N]:` Press **Y**. Only appears if you ran it as an administrator.
- `Attempt a complete WinGet repair now? [Y/N]:` Press **Y**. Same.

Waiting. `Updating MSYS2 packages...` and `Installing the UCRT64 GCC toolchain...` take 5 to 15 minutes, depending on your internet. A second window may open and close by itself. Do not touch anything.

The end. When it is done, it prints:

```
GCC installation completed successfully.
```

With Ways 1 and 3, **the black window then closes by itself**. That is normal. If it closes within a few seconds of starting, something went wrong: use Way 2, whose window stays open, and read the last lines. With Way 2 the window stays open and you can close it yourself.

Close every open terminal and every VS Code window. This is important. A terminal that was open *before* the installer ran does not know about the new compiler. Close them all.

Check. Open a new terminal: press the Windows key, type `powershell`, press Enter. Type:

```
gcc --version
```

and press Enter. You should see a line starting with `gcc.exe` and a version number, for example `gcc.exe (Rev2, Built by MSYS2 project) 15.2.0`. The numbers may differ. If you see that, you are done with the compiler. Go to section 8.

What the installer changed on your computer. It put the compiler in the folder `C:\Users\<your name>\AppData\Local\msys64` (or `C:\msys64` if you ran it as administrator). It added the folder `...\msys64\ucrt64\bin` to your PATH, which is the list of places Windows looks for programs. Nothing else.

If it did not work. Run it with Way 2. The messages stay on the screen, and the last lines say what went wrong. Take a photo of them, or select the text and copy it, and post it in the forum.

If you see `gcc is not recognized` after a successful install, you have not closed the old terminal. Close all terminals and VS Code, open a new terminal, try again.

3. Windows, option B: WSL

WSL puts a small Linux inside your Windows. You then install the compiler the Linux way. This option needs administrator rights and one restart.

1. Press the Windows key, type `powershell`, **right-click** "Windows PowerShell" and choose **Run as administrator**. Click **Yes**.
2. Type this and press Enter:

   ```
   wsl --install
   ```

1. Wait until it says it is done. Then **restart your computer**.
2. After the restart, a window called **Ubuntu** opens by itself. (If not, press the Windows key, type `Ubuntu`, press Enter.) It asks for a **username**: type a short name with no spaces, all lowercase, and press Enter. It asks for a **password**: type one and press Enter. **You will not see anything while typing the password.** Not even dots. That is normal. Type it again when asked.
3. You are now in a Linux terminal. Type these two lines, one at a time, and press Enter after each. The second one asks for your password.

   ```
   sudo apt update
   sudo apt install build-essential
   ```

When it asks `Do you want to continue? [Y/n]`, press **Y** and Enter.

1. Check:
   ```
   gcc --version
   ```

You should see `gcc (Ubuntu ...)` and a version number.

1. In VS Code (section 8), install the extension called **WSL** by Microsoft. Then, in the Ubuntu window, go to your folder and type `code .` (with the dot). VS Code opens that folder, and its terminal is the Linux terminal.

(WSL on this machine is installed but has no Linux inside it, so steps 3 to 7 could not be re-run here. They follow Microsoft's instructions. If a step looks different, post it in the forum.)

One thing to remember with WSL: when you use math functions, add `-lm` at the end of the compile command (see section 8).

4. Windows, option C: Visual Studio

Microsoft's own compiler is called **MSVC**. Its command is `cl`, not `gcc`. It accepts the C we use in this course. Choose this only if you already have Visual Studio, or you want it for another course. It is big.

1. Go to https://visualstudio.microsoft.com/ and download **Visual Studio Community**. It is free. (If you only want the compiler, the smaller **Build Tools for Visual Studio** on the same site also works.)
2. Run the installer. When it shows the list of **Workloads**, tick **Desktop development with C++**. Click **Install**. This takes a long time.
3. When it is done, press the Windows key and type `Developer Command Prompt`. Open **Developer Command Prompt for VS 2022**. This is a terminal that knows where `cl` is. A normal terminal does not.
4. In that window, go to your folder (`cd` and the folder) and compile:
   ```
   cl hello.c
   ```

This makes `hello.exe`. Run it by typing `hello` and Enter.

How `cl` is different from `gcc`.

- There is no `-o`. `cl hello.c` always makes `hello.exe`.
- Warnings: use `cl /W4 hello.c` instead of `gcc -Wall`.
- Error messages look different. `gcc` says `hello.c:3:28: error: expected ';'`. `cl` says something like `hello.c(3): error C2143: syntax error : missing ';' before 'return'`. Same meaning: file, line, what is missing.
- You do not need `-lm` for math functions.

(Not tested on this machine. Visual Studio is not installed here.)

5. Mac

1. Open **Terminal**: press **Cmd** + **Space**, type `Terminal`, press Enter.
2. Type this and press Enter:
   ```
   xcode-select --install
   ```

1. A window pops up asking to install the command line developer tools. Click **Install**, then **Agree**. Wait. It can take 10 minutes.

2. Check:

```
clang --version
```

You should see `Apple clang version ...`.

On a Mac the compiler is called **Clang**, not GCC. That is fine. It works exactly the same for this course. You can even type `gcc` on a Mac: it runs Clang. Use `gcc -Wall hello.c -o hello` like everybody else.

If `xcode-select --install` says the tools are already installed, you are done.

6. Linux

Open a terminal and type the line for your system. It asks for your password.

- Ubuntu, Debian, Linux Mint:

```
sudo apt update
sudo apt install build-essential
```

- Fedora:

```
sudo dnf install gcc
```

- Arch:

```
sudo pacman -S gcc
```

Check with `gcc --version`.

On Linux, when a program uses math functions (`sqrt`, `pow` and so on), add `-lm` at the end of the compile command. See section 8.

7. Any computer, nothing to install: CoCalc

CoCalc is a website with Linux and a C compiler already inside. You write and run your programs in the browser. It is what we used in the first class.

1. Go to https://cocalc.ai/ and click **Sign up** (top right).
2. Choose **Continue with Google** and pick your Google account, or type your email address and click **Continue with email**, then follow the steps it shows.
3. The free plan is enough for this course. You do not need a credit card. Do not start a paid trial.
4. Create a project: click the button to create a new project, give it a name like `c-course`, and open it.
5. Inside the project, create a file: click **New**, type the name `hello.c`, and choose to create it. An editor opens. Type your program there. It saves by itself.
6. Open a terminal: click **New** again and choose **Terminal**. A black box opens in the browser. That is your terminal, and the commands are exactly the ones in this guide and in class.
7. In the terminal, type `gcc -Wall hello.c -o hello`, Enter, then `./hello`, Enter.

(The exact names of the buttons inside a project were not checked on 2026-09-11; the sign-up page and the free plan were. If a button is called something else, look for "New" and "Terminal".)

Two things about the free plan: it can be slow at busy times, and your project goes to sleep when you close the browser. Nothing is lost. Just open it again.

8. VS Code, the editor

Everybody does this part.

Install it.

1. Go to https://code.visualstudio.com/ and click the big **Download** button. Run what you downloaded.
2. On Windows, click **Next** through the installer. On the page with tick boxes, leave **Add to PATH** ticked. On a Mac, drag the app into *Applications*.

Install the C/C++ extension. This colours your code and underlines mistakes.

1. Open VS Code.
2. Click the **Extensions** icon in the left bar. It looks like four small squares. Or press **Ctrl + Shift + X** (Mac: **Cmd + Shift + X**).
3. In the search box, type `C/C++`.
4. The first result is **C/C++** by **Microsoft**. Click **Install**.

Open your folder.

1. Click **File**, then **Open Folder**. Choose or create the folder for your programs, for example `c_week1`. Click **Select Folder**. If VS Code asks whether you trust the authors, click **Yes**.
2. To make a new file: click **File**, then **New Text File**. Type your program. Click **File**, then **Save As**, and name it `hello.c`. The `.c` at the end matters.

Open the terminal inside VS Code.

Click **View**, then **Terminal**. A panel opens at the bottom. That is a terminal, already inside your folder. You do not need any other terminal window.

Compile and run. In that terminal, type these two lines, one at a time, pressing Enter after each:

```
gcc -Wall hello.c -o hello
./hello
```

On Windows the terminal is PowerShell, and the second line is:

```
.\hello
```

That is a backslash `\`, not a slash `/`. If `./hello` says something like "not recognized", you are on Windows: use `.\hello`.

If your program uses math functions (`#include <math.h>`, `sqrt`, `pow`), then on Linux and WSL the compile line needs `-lm` at the end:

```
gcc -Wall hello.c -o hello -lm
```

On Windows with the course installer, and on a Mac, `-lm` is not needed, but it does no harm.

9. Check that it works

Type this program into VS Code and save it as `hello.c`:

```
#include <stdio.h>
int main(void) {
    printf("Hello, ADA!\n");
    return 0;
```

```
}
```

In the terminal:

```
gcc -Wall hello.c -o hello
./hello
```

After the first line you should see **nothing**. Silence means success. After the second line you should see:

```
Hello, ADA!
```

If you see that, you are ready.

The four most common problems.

1. `gcc : The term 'gcc' is not recognized` or **`gcc: command not found`**. The terminal cannot find the compiler.

- Windows: you opened this terminal *before* the installer ran. Close **all** terminals and VS Code, open a new one, try again. If it still fails, the installer did not finish; see "If it did not work" in section 2.
- Mac: run `xcode-select --install` from section 5.
- Linux: run the install command from section 6.

2. `hello.c: No such file or directory`. The compiler cannot see your file. You are in the wrong folder. Type `ls` (Windows: `dir`) and press Enter. If `hello.c` is not in the list, you are in the wrong place. In VS Code, click **File**, **Open Folder** and open the folder where `hello.c` is, then open the terminal again.

3. `permission denied` when you try to run the program on a Mac or Linux. You typed `hello.c` instead of `hello`, or the compile step failed and there is no program yet. Compile again, then run `./hello`.

4. `undefined reference to 'sqrt'`. Your program uses math functions and you are on Linux or WSL. Add `-lm` to the end of the compile line.

And one that is not an error at all: after `gcc` you see nothing, and you wait for something to happen. Nothing will. Silence means it worked. Now run the program.

10. Where to ask

Post in the **Questions and answers** forum on Blackboard. Say what computer you have, which option you chose, which step you were on, and paste the exact message from the screen. A photo of the screen is fine too. Someone else has the same problem, and the answer helps everybody.

Or bring the laptop to class. We fix set-ups at the end of every class in the first two weeks.